

jenkins

To install jenkins manually

- 1.sudo apt update
- 2.sudo apt install openjdk-17-jdk -y
- 3.sudo apt update -y
- 4.sudo apt install maven -y
- 5.sudo wget -O /etc/apt/keyrings/jenkins-keyring.asc \
https://pkg.jenkins.io/debian-stable/jenkins.io-2026.key
- 6.echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc]" \
https://pkg.jenkins.io/debian-stable binary/ | sudo tee \
/etc/apt/sources.list.d/jenkins.list > /dev/null
- 7.sudo apt update
- 8.sudo apt install Jenkins
- 9.systemctl status Jenkins

Install Jenkins from git repository

wget <https://raw.githubusercontent.com/akshu20791/Deployment-script/refs/heads/main/jenkins.sh>

chmod 700 jenkins.sh

./jenkins.sh

#Use below lines if if different java versions are installed

update-java-alternatives -l

sudo update-alternatives --config java

select 1

java --version

sudo systemctl restart sonar

Tomcat

wget <https://dlcdn.apache.org/tomcat/tomcat-9/v9.0.115/bin/apache-tomcat-9.0.115.zip>

apt install unzip -y

unzip apache-tomcat-9.0.115.zip

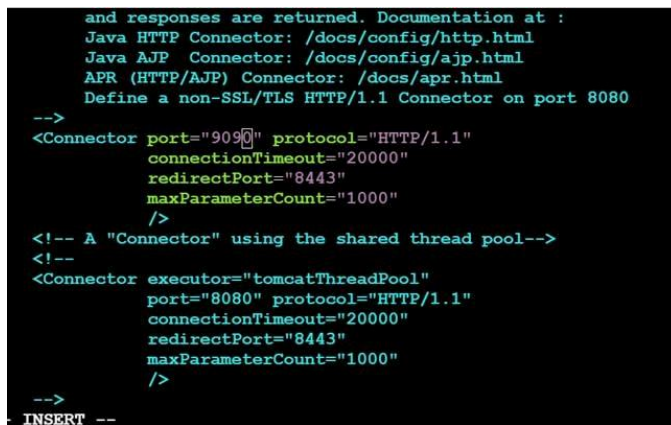
cd apache-tomcat-9.0.115

now we need to change the default port of tomcat from 8080 to 9090

cd conf

ls

vi server.xml



```
and responses are returned. Documentation at :
Java HTTP Connector: /docs/config/http.html
Java AJP Connector: /docs/config/ajp.html
APR (HTTP/AJP) Connector: /docs/apr.html
Define a non-SSL/TLS HTTP/1.1 Connector on port 8080
-->
<Connector port="9090" protocol="HTTP/1.1"
    connectionTimeout="20000"
    redirectPort="8443"
    maxParameterCount="1000"
/>
<!-- A "Connector" using the shared thread pool-->
<!--
<Connector executor="tomcatThreadPool"
    port="8080" protocol="HTTP/1.1"
    connectionTimeout="20000"
    redirectPort="8443"
    maxParameterCount="1000"
/>
-->
INSERT --
```

cd ..

cd bin

ls

chmod +x *.sh

./startup.sh

#visudo : this file gives the user the root (sudo) permission to perform a tasks...we will give jenkins the sudo permissions to perform the tasks

visudo

#add jenkins ALL=(ALL:ALL) NOPASSWD: ALL

```
GNU nano 7.2 /etc/sudoers.tmp *
# Ditto for GPG agent
#Defaults:%sudo env_keep += "GPG_AGENT_INFO"

# Host alias specification

# User alias specification

# Cmnd alias specification

# User privilege specification
root    ALL=(ALL:ALL) ALL
jenkins ALL=(ALL:ALL) NOPASSWD: ALL
# Members of the admin group may gain root privileges
%admin   ALL=(ALL) ALL

# Allow members of group sudo to execute any command

^G Help      ^O Write Out ^W Where Is  ^R Cut       ^T Execute  ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify  ^_ Go To Line
```

press ctrl x and y and then enter to come out of the nano editor

service jenkins restart

Sonarqube

<https://medium.com/@deshdeepakdhobi/how-to-install-and-configure-sonarqube-on-aws-ec2-ubuntu-22-04-c89a3f1c2447>

sudo apt update

sudo apt upgrade -y

sudo apt install -y openjdk-17-jdk

java -version

sudo sh -c 'echo "deb http://apt.postgresql.org/pub/repos/apt/ `lsb\_release -cs`-pgdg main" /etc/apt/sources.list.d/pgdg.list'

wget -q https://www.postgresql.org/media/keys/ACCC4CF8.asc -O - | sudo apt-key add -

sudo apt install postgresql postgresql-contrib -y

sudo systemctl enable postgresql

sudo systemctl start postgresql

sudo systemctl status postgresql

psql --version

sudo -i -u postgres

createuser ddsonar

psql

ALTER USER ddsonar WITH ENCRYPTED password '1234';

CREATE DATABASE ddsonarqube OWNER ddsonar;

```

GRANT ALL PRIVILEGES ON DATABASE ddsonarqube to ddsonar;

\l
\du
\q
exit

sudo apt install zip -y

sudo wget https://binaries.sonarsource.com/Distribution/sonarqube/sonarqube-10.0.0.68432.zip

sudo unzip sonarqube-10.0.0.68432.zip

sudo mv sonarqube-10.0.0.68432 sonarqube

sudo mv sonarqube /opt/

sudo groupadd ddsonar

sudo useradd -d /opt/sonarqube -g ddsonar ddsonar

sudo chown ddsonar:ddsonar /opt/sonarqube -R

sudo nano /opt/sonarqube/conf/sonar.properties

#Uncomment the lines, and add the database user and Database password

sonar.jdbc.username=ddsonar

sonar.jdbc.password=1234

#Below these two lines, add the following line of code.

sonar.jdbc.url=jdbc:postgresql://localhost:5432/ddsonarqube

sudo nano /opt/sonarqube/bin/linux-x86-64/sonar.sh

#Add the following line

RUN_AS_USER=ddsonar

sudo nano /etc/systemd/system/sonar.service

[Unit]
Description=SonarQube service
After=syslog.target network.target
[Service]
Type=forking
ExecStart=/opt/sonarqube/bin/linux-x86-64/sonar.sh start
ExecStop=/opt/sonarqube/bin/linux-x86-64/sonar.sh stop
User=ddsonar
Group=ddsonar
Restart=always
LimitNOFILE=65536
LimitNPROC=4096

```

[Install]

WantedBy=multi-user.target

#Paste above lines in sonar.service

sudo systemctl enable sonar

sudo systemctl start sonar

sudo systemctl status sonar

sudo nano /etc/sysctl.conf

#add these at the end

vm.max_map_count=262144

fs.file-max=65536

ulimit -n 65536

ulimit -u 4096

sudo reboot

10) Access SonarQube Web Interface

i) Access SonarQube in a web browser at your server's IP address on port 9000.

For example, <http://IP:9000>

Note: the default username and password are **admin** and **admin** respectively.

ii) Change the Old password with a New one.

Log in with username **admin** and password **admin**. In the next step, SonarQube will prompt you to change your password. **CHANGE THE PASSWORD.**

Sonar Scanner plugin download for sonarqube

Pipeline script

```
pipeline {
    agent any

    tools {
        maven 'mymaven'
    }

    stages {
        stage('Step 1 : Clone Repository') {
            steps {
                echo "Cloning repository from GitHub"

                git 'https://github.com/akshu20791/addressbook-cicd-project'
            }
        }
    }
}
```

```
stage('Step 2 : Compile Code') {
```

```
  steps {
```

```
    echo "Compiling the project"
```

```
    sh 'mvn clean compile'
```

```
  }
```

```
}
```

```
stage('Step 3 : Code Review (PMD)') {
```

```
  steps {
```

```
    echo "Running PMD code analysis"
```

```
    sh 'mvn pmd:pmd'
```

```
  }
```

```
  post {
```

```
    success {
```

```
      recordIssues tools: [pmdParser(pattern: '**/pmd.xml')]
```

```
    }
```

```
  }
```

```
}
```

```
stage('Step 4 : Unit Testing') {
```

```
  steps {
```

```
    echo "Running unit tests"
```

```
    sh 'mvn test'
```

```
  }
```

```
  post {
```

```
    success {
```

```
      junit 'target/surefire-reports/*.xml'
```

```
    }
```

```
  }
```

```
}
```

```
stage('Step 5 : SonarQube Analysis') {
```

```
  steps {
```

```
    echo "Starting SonarQube scan"
```

```
    withSonarQubeEnv('sonarqubeserver') {
        sh 'mvn clean verify sonar:sonar'
    }
}

stage('Step 6 : Package Application') {
    steps {
        echo "Packaging the application"
        sh 'mvn package'
    }
}

stage('Deployment of the App') {
    steps {
        sh "sudo mv ${WORKSPACE}/target/addressbook.war /home/ubuntu/apache-tomcat-9.0.115/webapps/"
    }
}

stage('Quality Gate') {
    steps {
        timeout(time: 10, unit: 'MINUTES') {
            waitForQualityGate abortPipeline: true
        }
    }
}
}
```